

Enhanced Repository Files for Cross-Platform AI Agent Implementation

Based on the analysis of your Enhanced Resume Agent 3.0 repository and the deployment recommendations, here are the complete, optimized files to make your repository ready for app and PWA implementations across Android, Chromebook, iMac, and iPad.

Enhanced Project Structure

```
enhanced-resume-agent/
├── backend/                                # API Backend (FastAPI/Flask)
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py                        # FastAPI main application
│   │   ├── models/                       # Pydantic models
│   │   ├── routes/                       # API route handlers
│   │   ├── services/                     # Business logic
│   │   └── utils/                         # Utility functions
│   ├── celery_app/                       # Celery task management
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/                              # Web frontend & PWA
│   ├── static/
│   │   ├── css/
│   │   ├── js/
│   │   ├── icons/
│   │   ├── manifest.json                 # PWA manifest
│   │   └── sw.js                         # Service worker
│   ├── templates/
│   └── index.html
├── mobile/                                # Mobile app preparations
│   ├── flutter/                          # Flutter app structure
│   ├── react-native/                    # React Native app structure
│   └── api-client/                       # Generated API clients
├── docs/                                  # Enhanced documentation
│   ├── api/                             # API documentation
│   ├── deployment/                      # Deployment guides
│   └── development/                     # Development guides
├── docker-compose.yml                    # Updated for single-container deployment
├── .env.example                          # Environment variables template
├── langgraph.json                        # LangGraph configuration
└── README.md                             # Enhanced README
```

1. Enhanced README.md

```
# Enhanced Resume Agent 3.0 – Cross-Platform Personal Edition
```

A sophisticated AI-powered career toolkit that generates tailored resumes, cover letters,

```
## ✨ Features
```

- **AI-Powered Generation**: Uses Google Gemini for intelligent document creation
- **Cross-Platform**: Web, PWA, Flutter, and React Native ready
- **State Machine**: LangGraph-powered workflow orchestration
- **Job Scouting**: Automated Gmail-based job discovery
- **Vector Database**: Pluggable storage with multiple backends
- **Secure Authentication**: OAuth2/JWT with Google integration
- **RESTful API**: OpenAPI-documented endpoints
- **Real-time Updates**: WebSocket support for streaming responses

```
## 🚀 Quick Start
```

```
### Prerequisites
```

- Python 3.9+
- Docker & Docker Compose
- Google Gemini API key
- Gmail API credentials

```
### 1. Environment Setup
```

```
cp .env.example .env
```

Edit .env with your API keys and configuration

```
### 2. Docker Deployment (Recommended)
```

```
docker-compose up --build
```

```
### 3. Local Development
```

```
pip install -r backend/requirements.txt
```

```
uvicorn backend.app.main:app --reload
```

```
## 📍 Access Points
```

- **Web Application**: `http://localhost:8000`
- **API Documentation**: `http://localhost:8000/docs`
- **PWA**: Install via browser from web application
- **Mobile Apps**: See `mobile/` directory for Flutter/React Native

```
## 🏗️ Architecture
```

```

#### Backend (FastAPI)
- **API Layer**: RESTful endpoints with OpenAPI documentation
- **Business Logic**: LangGraph state machines for document generation
- **Task Queue**: Celery for background job processing
- **Authentication**: JWT tokens with Google OAuth2

#### Frontend Options
- **Web/PWA**: Progressive Web App with offline support
- **Flutter**: Native mobile app for iOS/Android
- **React Native**: Alternative mobile framework

## 📦 PWA Features

- **Offline Support**: Service worker caching
- **Install Prompt**: Add to home screen
- **Push Notifications**: Background updates
- **Responsive Design**: Works on all devices

## 📡 API Endpoints

#### Authentication
- `POST /auth/login` - User login
- `POST /auth/refresh` - Token refresh
- `POST /auth/logout` - User logout

#### Document Generation
- `POST /api/generate/resume` - Generate resume
- `POST /api/generate/cover-letter` - Generate cover letter
- `POST /api/generate/criteria` - Generate selection criteria

#### Job Management
- `GET /api/jobs` - List scraped jobs
- `POST /api/jobs/search` - Search jobs
- `PUT /api/jobs/{id}/apply` - Mark job as applied

## 🚀 Deployment Options

#### Cloud Run (Recommended)

```

```

gcloud builds submit --tag gcr.io/
PROJECT_ID/resume --agent gcloudrundeploy --image gcr.io/
PROJECT_ID/resume-agent

```

```

#### Other Platforms
- **Fly.io**: See `docs/deployment/fly.md`
- **Vercel**: See `docs/deployment/vercel.md`
- **Railway**: See `docs/deployment/railway.md`

## 📖 Documentation

- [API Documentation](docs/api/README.md)
- [Deployment Guide](docs/deployment/README.md)
- [Development Setup](docs/development/README.md)

```

```
- [Mobile App Development](docs/mobile/README.md)
```

```
## 📁 Development
```

```
### Backend Development
```

```
cd backend
```

```
pip install -r requirements.txt
```

```
uvicorn app.main:app --reload
```

```
### Frontend Development
```

```
cd frontend
```

```
python -m http.server 8080
```

```
### Mobile Development
```

Flutter

```
cd mobile/flutter
```

```
flutter run
```

React Native

```
cd mobile/react-native
```

```
npm start
```

```
## 🛡 Security
```

- JWT-based authentication
- CORS configuration
- Rate limiting
- Input validation
- Secure headers

```
## 🤝 Contributing
```

1. Fork the repository
2. Create your feature branch
3. Commit your changes
4. Push to the branch
5. Create a Pull Request

```
## 📄 License
```

This project is licensed under the MIT License.

2. Enhanced Backend Structure

backend/app/main.py

```
"""
Enhanced Resume Agent 3.0 - FastAPI Backend
Cross-platform ready with OpenAPI documentation
"""

from fastapi import FastAPI, HTTPException, Depends, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
from fastapi.responses import HTMLResponse
import uvicorn
import os
from pathlib import Path

from app.routes import auth, documents, jobs, admin
from app.models.base import ResponseModel
from app.utils.auth import verify_jwt_token
from app.utils.database import init_db

# Initialize FastAPI app
app = FastAPI(
    title="Enhanced Resume Agent 3.0 API",
    description="AI-powered career toolkit with cross-platform support",
    version="3.0.0",
    docs_url="/docs",
    redoc_url="/redoc",
    openapi_url="/openapi.json"
)

# Security scheme
security = HTTPBearer()

# CORS configuration for cross-platform access
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # Configure for production
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Mount static files for PWA
app.mount("/static", StaticFiles(directory="frontend/static"), name="static")

# Include routers
app.include_router(auth.router, prefix="/auth", tags=["Authentication"])
app.include_router(documents.router, prefix="/api/documents", tags=["Documents"])
app.include_router(jobs.router, prefix="/api/jobs", tags=["Jobs"])
app.include_router(admin.router, prefix="/api/admin", tags=["Admin"])

@app.on_event("startup")
```

```

async def startup_event():
    """Initialize database and services on startup"""
    await init_db()
    print("Enhanced Resume Agent 3.0 started successfully!")

@app.get("/", response_class=HTMLResponse)
async def root():
    """Serve the main PWA application"""
    html_file = Path("frontend/templates/index.html")
    if html_file.exists():
        return HTMLResponse(content=html_file.read_text(), status_code=200)
    return {"message": "Enhanced Resume Agent 3.0 API", "docs": "/docs"}

@app.get("/health", response_model=ResponseModel)
async def health_check():
    """Health check endpoint"""
    return ResponseModel(
        success=True,
        message="Service is healthy",
        data={"version": "3.0.0", "status": "online"}
    )

# Dependency for protected routes
async def get_current_user(credentials: HTTPAuthorizationCredentials = Depends(security)):
    """Verify JWT token and return user"""
    token = credentials.credentials
    user = await verify_jwt_token(token)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid authentication credentials",
            headers={"WWW-Authenticate": "Bearer"},
        )
    return user

if __name__ == "__main__":
    uvicorn.run(
        "app.main:app",
        host="0.0.0.0",
        port=8000,
        reload=True,
        log_level="info"
    )

```

backend/app/models/base.py

```

"""
Base models for API responses and requests
"""

from pydantic import BaseModel, Field
from typing import Optional, Any, Dict, List
from datetime import datetime
from enum import Enum

```

```

class ResponseModel(BaseModel):
    """Standard API response model"""
    success: bool = Field(..., description="Whether the request was successful")
    message: str = Field(..., description="Human-readable message")
    data: Optional[Any] = Field(None, description="Response data")
    errors: Optional[List[str]] = Field(None, description="List of errors")
    timestamp: datetime = Field(default_factory=datetime.utcnow)

class DocumentType(str, Enum):
    """Document type enumeration"""
    RESUME = "resume"
    COVER_LETTER = "cover_letter"
    CRITERIA = "criteria"

class JobStatus(str, Enum):
    """Job status enumeration"""
    DISCOVERED = "discovered"
    REVIEWING = "reviewing"
    APPLIED = "applied"
    INTERVIEW = "interview"
    REJECTED = "rejected"

class UserProfile(BaseModel):
    """User profile model"""
    user_id: str = Field(..., description="User identifier")
    email: str = Field(..., description="User email")
    full_name: str = Field(..., description="User full name")
    skills: List[str] = Field(default_factory=list, description="User skills")
    experience: str = Field("", description="User experience")
    preferences: Dict[str, Any] = Field(default_factory=dict)
    created_at: datetime = Field(default_factory=datetime.utcnow)
    updated_at: datetime = Field(default_factory=datetime.utcnow)

class DocumentRequest(BaseModel):
    """Document generation request"""
    document_type: DocumentType = Field(..., description="Type of document to generate")
    job_description: str = Field(..., description="Job description")
    tone: str = Field("professional", description="Tone of voice")
    additional_requirements: Optional[str] = Field(None, description="Additional requirements")
    template_id: Optional[str] = Field(None, description="Template identifier")

class DocumentResponse(BaseModel):
    """Document generation response"""
    document_id: str = Field(..., description="Generated document ID")
    document_type: DocumentType = Field(..., description="Type of document")
    content: str = Field(..., description="Generated content")
    metadata: Dict[str, Any] = Field(default_factory=dict)
    created_at: datetime = Field(default_factory=datetime.utcnow)

class JobListing(BaseModel):
    """Job listing model"""
    job_id: str = Field(..., description="Job identifier")
    title: str = Field(..., description="Job title")
    company: str = Field(..., description="Company name")
    description: str = Field(..., description="Job description")
    requirements: List[str] = Field(default_factory=list)

```

```

salary_range: Optional[str] = Field(None, description="Salary range")
location: str = Field(..., description="Job location")
source: str = Field(..., description="Source of job listing")
status: JobStatus = Field(default=JobStatus.DISCOVERED)
discovered_at: datetime = Field(default_factory=datetime.utcnow)
applied_at: Optional[datetime] = Field(None)

```

backend/app/routes/documents.py

```

"""
Document generation API routes
"""

from fastapi import APIRouter, Depends, HTTPException, BackgroundTasks
from fastapi.responses import StreamingResponse
from typing import List, Optional
import json
import asyncio

from app.models.base import ResponseModel, DocumentRequest, DocumentResponse
from app.services.document_generator import DocumentGeneratorService
from app.services.langgraph_orchestrator import LangGraphOrchestrator
from app.utils.auth import get_current_user
from app.utils.streaming import generate_stream_response

router = APIRouter()

@router.post("/generate", response_model=ResponseModel)
async def generate_document(
    request: DocumentRequest,
    background_tasks: BackgroundTasks,
    current_user: dict = Depends(get_current_user)
):
    """Generate a document using AI"""
    try:
        # Initialize services
        generator = DocumentGeneratorService()
        orchestrator = LangGraphOrchestrator()

        # Start document generation
        result = await orchestrator.generate_document(
            user_id=current_user["user_id"],
            request=request
        )

        # Add background task for post-processing
        background_tasks.add_task(
            generator.post_process_document,
            result["document_id"]
        )

    return ResponseModel(
        success=True,
        message="Document generated successfully",
        data=result
    )

```



```

    )
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/generate/stream/{document_id}")
async def stream_document_generation(
    document_id: str,
    current_user: dict = Depends(get_current_user)
):
    """Stream document generation progress"""
    try:
        orchestrator = LangGraphOrchestrator()

        async def generate():
            async for chunk in orchestrator.stream_generation(document_id):
                yield f"data: {json.dumps(chunk)}\n\n"

        return StreamingResponse(
            generate(),
            media_type="text/stream",
            headers={
                "Cache-Control": "no-cache",
                "Connection": "keep-alive",
            }
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/", response_model=ResponseModel)
async def get_documents(
    document_type: Optional[str] = None,
    limit: int = 10,
    offset: int = 0,
    current_user: dict = Depends(get_current_user)
):
    """Get user's documents"""
    try:
        service = DocumentGeneratorService()
        documents = await service.get_user_documents(
            user_id=current_user["user_id"],
            document_type=document_type,
            limit=limit,
            offset=offset
        )

        return ResponseModel(
            success=True,
            message="Documents retrieved successfully",
            data=documents
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/{document_id}", response_model=ResponseModel)
async def get_document(
    document_id: str,

```

```

        current_user: dict = Depends(get_current_user)
    ):
        """Get a specific document"""
        try:
            service = DocumentGeneratorService()
            document = await service.get_document(document_id)

            if not document or document.get("user_id") != current_user["user_id"]:
                raise HTTPException(status_code=404, detail="Document not found")

            return ResponseModel(
                success=True,
                message="Document retrieved successfully",
                data=document
            )
        except Exception as e:
            raise HTTPException(status_code=500, detail=str(e))

    @router.put("/{document_id}", response_model=ResponseModel)
    async def update_document(
        document_id: str,
        updates: dict,
        current_user: dict = Depends(get_current_user)
    ):
        """Update a document"""
        try:
            service = DocumentGeneratorService()
            document = await service.update_document(
                document_id=document_id,
                user_id=current_user["user_id"],
                updates=updates
            )

            return ResponseModel(
                success=True,
                message="Document updated successfully",
                data=document
            )
        except Exception as e:
            raise HTTPException(status_code=500, detail=str(e))

    @router.delete("/{document_id}", response_model=ResponseModel)
    async def delete_document(
        document_id: str,
        current_user: dict = Depends(get_current_user)
    ):
        """Delete a document"""
        try:
            service = DocumentGeneratorService()
            await service.delete_document(
                document_id=document_id,
                user_id=current_user["user_id"]
            )

            return ResponseModel(
                success=True,

```

```
        message="Document deleted successfully"
    )
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))
```

3. PWA Configuration Files

frontend/static/manifest.json

```
{
  "name": "Enhanced Resume Agent 3.0",
  "short_name": "Resume Agent",
  "description": "AI-powered career toolkit for resume generation and job scouting",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#007bff",
  "orientation": "portrait-primary",
  "categories": ["productivity", "business", "utilities"],
  "lang": "en-US",
  "dir": "ltr",
  "scope": "/",
  "icons": [
    {
      "src": "/static/icons/icon-72x72.png",
      "sizes": "72x72",
      "type": "image/png",
      "purpose": "maskable any"
    },
    {
      "src": "/static/icons/icon-96x96.png",
      "sizes": "96x96",
      "type": "image/png",
      "purpose": "maskable any"
    },
    {
      "src": "/static/icons/icon-128x128.png",
      "sizes": "128x128",
      "type": "image/png",
      "purpose": "maskable any"
    },
    {
      "src": "/static/icons/icon-144x144.png",
      "sizes": "144x144",
      "type": "image/png",
      "purpose": "maskable any"
    },
    {
      "src": "/static/icons/icon-152x152.png",
      "sizes": "152x152",
      "type": "image/png",
      "purpose": "maskable any"
    },
    {

```

```
    "src": "/static/icons/icon-192x192.png",
    "sizes": "192x192",
    "type": "image/png",
    "purpose": "maskable any"
  },
  {
    "src": "/static/icons/icon-384x384.png",
    "sizes": "384x384",
    "type": "image/png",
    "purpose": "maskable any"
  },
  {
    "src": "/static/icons/icon-512x512.png",
    "sizes": "512x512",
    "type": "image/png",
    "purpose": "maskable any"
  }
],
"shortcuts": [
  {
    "name": "Generate Resume",
    "url": "/generate?type=resume",
    "description": "Quick resume generation"
  },
  {
    "name": "Job Search",
    "url": "/jobs",
    "description": "Browse discovered jobs"
  }
],
"screenshots": [
  {
    "src": "/static/screenshots/desktop-main.png",
    "sizes": "1280x720",
    "type": "image/png",
    "form_factor": "wide"
  },
  {
    "src": "/static/screenshots/mobile-main.png",
    "sizes": "360x640",
    "type": "image/png",
    "form_factor": "narrow"
  }
],
"related_applications": [
  {
    "platform": "play",
    "url": "https://play.google.com/store/apps/details?id=com.resumeagent.app"
  }
],
"prefer_related_applications": false,
"edge_side_panel": {
  "preferred_width": 400
}
}
```

frontend/static/sw.js

```
/**
 * Service Worker for Enhanced Resume Agent 3.0
 * Provides offline functionality and caching
 */

const CACHE_NAME = 'resume-agent-v3.0.0';
const API_CACHE_NAME = 'resume-agent-api-v3.0.0';

// Files to cache for offline functionality
const STATIC_CACHE_FILES = [
  '/',
  '/static/css/main.css',
  '/static/js/main.js',
  '/static/icons/icon-192x192.png',
  '/static/icons/icon-512x512.png',
  '/offline.html'
];

// API endpoints to cache
const API_CACHE_PATTERNS = [
  /\api\/documents/,
  /\api\/jobs/,
  /\api\/user/
];

// Install event - cache static assets
self.addEventListener('install', (event) => {
  console.log('Service Worker: Installing...');
  event.waitUntil(
    caches.open(CACHE_NAME)
      .then((cache) => {
        console.log('Service Worker: Caching static files');
        return cache.addAll(STATIC_CACHE_FILES);
      })
      .then(() => {
        console.log('Service Worker: Installed successfully');
        return self.skipWaiting();
      })
  );
});

// Activate event - clean up old caches
self.addEventListener('activate', (event) => {
  console.log('Service Worker: Activating...');
  event.waitUntil(
    caches.keys().then((cacheNames) => {
      return Promise.all(
        cacheNames.map((cacheName) => {
          if (cacheName !== CACHE_NAME && cacheName !== API_CACHE_NAME) {
            console.log('Service Worker: Deleting old cache:', cacheName);
            return caches.delete(cacheName);
          }
        })
      );
    })
  );
});
```

```

    }).then(() => {
      console.log('Service Worker: Activated successfully');
      return self.clients.claim();
    })
  );
});

// Fetch event - handle requests
self.addEventListener('fetch', (event) => {
  const { request } = event;
  const url = new URL(request.url);

  // Handle API requests
  if (url.pathname.startsWith('/api/')) {
    event.respondWith(handleApiRequest(request));
    return;
  }

  // Handle static file requests
  if (request.method === 'GET') {
    event.respondWith(handleStaticRequest(request));
    return;
  }
});

// Handle API requests with network-first strategy
async function handleApiRequest(request) {
  try {
    // Try network first
    const networkResponse = await fetch(request);

    // Cache successful GET requests
    if (request.method === 'GET' && networkResponse.ok) {
      const cache = await caches.open(API_CACHE_NAME);
      cache.put(request, networkResponse.clone());
    }

    return networkResponse;
  } catch (error) {
    // Fallback to cache for GET requests
    if (request.method === 'GET') {
      const cache = await caches.open(API_CACHE_NAME);
      const cachedResponse = await cache.match(request);

      if (cachedResponse) {
        return cachedResponse;
      }
    }

    // Return offline page for failed requests
    return new Response(
      JSON.stringify({
        success: false,
        message: 'Network error - please check your connection',
        offline: true
      }),

```

```

    {
      status: 503,
      headers: {
        'Content-Type': 'application/json'
      }
    }
  );
}
}

// Handle static requests with cache-first strategy
async function handleStaticRequest(request) {
  try {
    const cache = await caches.open(CACHE_NAME);
    const cachedResponse = await cache.match(request);

    if (cachedResponse) {
      return cachedResponse;
    }

    const networkResponse = await fetch(request);

    if (networkResponse.ok) {
      cache.put(request, networkResponse.clone());
    }

    return networkResponse;
  } catch (error) {
    // Return offline page for navigation requests
    if (request.mode === 'navigate') {
      const cache = await caches.open(CACHE_NAME);
      return cache.match('/offline.html');
    }

    return new Response('Offline', { status: 503 });
  }
}

// Background sync for offline actions
self.addEventListener('sync', (event) => {
  if (event.tag === 'background-sync') {
    event.waitUntil(handleBackgroundSync());
  }
});

async function handleBackgroundSync() {
  // Handle queued actions when back online
  const cache = await caches.open('offline-actions');
  const requests = await cache.keys();

  for (const request of requests) {
    try {
      await fetch(request);
      await cache.delete(request);
    } catch (error) {
      console.log('Background sync failed for:', request.url);
    }
  }
}

```

```

    }
  }
}

// Push notification handler
self.addListener('push', (event) => {
  const options = {
    body: event.data ? event.data.text() : 'New job opportunities available!',
    icon: '/static/icons/icon-192x192.png',
    badge: '/static/icons/badge-72x72.png',
    vibrate: [100, 50, 100],
    data: {
      dateOfArrival: Date.now(),
      primaryKey: 1
    },
    actions: [
      {
        action: 'explore',
        title: 'View Jobs',
        icon: '/static/icons/action-explore.png'
      },
      {
        action: 'close',
        title: 'Close',
        icon: '/static/icons/action-close.png'
      }
    ]
  };

  event.waitUntil(
    self.registration.showNotification('Resume Agent', options)
  );
});

// Notification click handler
self.addListener('notificationclick', (event) => {
  event.notification.close();

  if (event.action === 'explore') {
    event.waitUntil(
      clients.openWindow('/jobs')
    );
  }
});
});

```

4. Mobile App Structure

mobile/flutter/lib/main.dart

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:http/http.dart' as http;
import 'services/api_service.dart';
import 'providers/auth_provider.dart';

```



```

import 'providers/document_provider.dart';
import 'screens/login_screen.dart';
import 'screens/home_screen.dart';
import 'screens/document_generator_screen.dart';
import 'screens/jobs_screen.dart';
import 'utils/theme.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) => AuthProvider()),
        ChangeNotifierProvider(create: (_) => DocumentProvider()),
      ],
      child: MaterialApp(
        title: 'Resume Agent 3.0',
        theme: AppTheme.lightTheme,
        darkTheme: AppTheme.darkTheme,
        themeMode: ThemeMode.system,
        initialRoute: '/login',
        routes: {
          '/login': (context) => LoginScreen(),
          '/home': (context) => HomeScreen(),
          '/generate': (context) => DocumentGeneratorScreen(),
          '/jobs': (context) => JobsScreen(),
        },
      ),
    );
  }
}

```

mobile/flutter/lib/services/api_service.dart

```

import 'dart:convert';
import 'package:http/http.dart' as http;
import '../models/document.dart';
import '../models/job.dart';
import '../models/user.dart';

class ApiService {
  static const String baseUrl = 'https://your-api-domain.com';

  String? _token;

  void setToken(String token) {
    _token = token;
  }

  Map<String, String> get _headers => {
    'Content-Type': 'application/json',
  }
}

```

```

    if (_token != null) 'Authorization': 'Bearer $_token',
};

// Authentication
Future<Map<String, dynamic>> login(String email, String password) async {
    final response = await http.post(
        Uri.parse('$baseUrl/auth/login'),
        headers: _headers,
        body: jsonEncode({
            'email': email,
            'password': password,
        }),
    );

    if (response.statusCode == 200) {
        return jsonDecode(response.body);
    } else {
        throw Exception('Login failed');
    }
}

// Document Generation
Future<Document> generateDocument({
    required String documentType,
    required String jobDescription,
    String tone = 'professional',
    String? additionalRequirements,
}) async {
    final response = await http.post(
        Uri.parse('$baseUrl/api/documents/generate'),
        headers: _headers,
        body: jsonEncode({
            'document_type': documentType,
            'job_description': jobDescription,
            'tone': tone,
            'additional_requirements': additionalRequirements,
        }),
    );

    if (response.statusCode == 200) {
        final data = jsonDecode(response.body);
        return Document.fromJson(data['data']);
    } else {
        throw Exception('Document generation failed');
    }
}

// Stream document generation
Stream<Map<String, dynamic>> streamDocumentGeneration(String documentId) async* {
    final response = await http.get(
        Uri.parse('$baseUrl/api/documents/generate/stream/$documentId'),
        headers: _headers,
    );

    if (response.statusCode == 200) {
        final lines = response.body.split('\n');

```

```

        for (final line in lines) {
            if (line.startsWith('data: ')) {
                final data = line.substring(6);
                yield jsonDecode(data);
            }
        }
    }
}

// Get user documents
Future<List<Document>> getDocuments({
    String? documentType,
    int limit = 10,
    int offset = 0,
}) async {
    final queryParams = <String, String>{
        'limit': limit.toString(),
        'offset': offset.toString(),
    };

    if (documentType != null) {
        queryParams['document_type'] = documentType;
    }

    final uri = Uri.parse('$baseUrl/api/documents').replace(
        queryParameters: queryParams,
    );

    final response = await http.get(uri, headers: _headers);

    if (response.statusCode == 200) {
        final data = jsonDecode(response.body);
        return (data['data'] as List)
            .map((json) => Document.fromJson(json))
            .toList();
    } else {
        throw Exception('Failed to load documents');
    }
}

// Get jobs
Future<List<Job>> getJobs({
    String? status,
    int limit = 10,
    int offset = 0,
}) async {
    final queryParams = <String, String>{
        'limit': limit.toString(),
        'offset': offset.toString(),
    };

    if (status != null) {
        queryParams['status'] = status;
    }

    final uri = Uri.parse('$baseUrl/api/jobs').replace(

```

```

        queryParameters: queryParams,
    );

    final response = await http.get(uri, headers: _headers);

    if (response.statusCode == 200) {
        final data = jsonDecode(response.body);
        return (data['data'] as List)
            .map((json) => Job.fromJson(json))
            .toList();
    } else {
        throw Exception('Failed to load jobs');
    }
}
}

```

5. Enhanced Documentation

docs/api/README.md

```

# API Documentation

### Overview

The Enhanced Resume Agent 3.0 API provides RESTful endpoints for document generation, job

### Base URL

- **Production**: `https://your-domain.com`
- **Development**: `http://localhost:8000`

### Authentication

The API uses JWT (JSON Web Tokens) for authentication. Include the token in the Authoriza

```

Authorization: Bearer <your_jwt_token>

```

### Obtaining a Token

```

```

curl -X POST
https://your-domain.com/auth/login
-H 'Content-Type: application/json'
-d '{
  "email": "user@example.com",
  "password": "your_password"
}'

```

API Endpoints

Authentication

POST /auth/login

Authenticate user and receive JWT token.

****Request Body:****

```
{
  "email": "user@example.com",
  "password": "your_password"
}
```

****Response:****

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "token": "jwt_token_here",
    "user": {
      "user_id": "user_123",
      "email": "user@example.com",
      "full_name": "John Doe"
    }
  }
}
```

POST /auth/refresh

Refresh JWT token.

POST /auth/logout

Logout user and invalidate token.

Document Generation

POST /api/documents/generate

Generate a new document using AI.

****Request Body:****

```
{
  "document_type": "resume",
  "job_description": "Software Engineer position...",
}
```

```
"tone": "professional",
"additional_requirements": "Emphasize Python skills"
}
```

****Response:****

```
{
  "success": true,
  "message": "Document generated successfully",
  "data": {
    "document_id": "doc_123",
    "document_type": "resume",
    "content": "Generated resume content...",
    "created_at": "2024-01-15T10:30:00Z"
  }
}
```

GET /api/documents/generate/stream/{document_id}
Stream document generation progress in real-time.

****Response:**** Server-Sent Events (SSE)

```
data: {"status": "starting", "progress": 0}
data: {"status": "analyzing", "progress": 25}
data: {"status": "generating", "progress": 50}
data: {"status": "formatting", "progress": 75}
data: {"status": "complete", "progress": 100, "document_id": "doc_123"}
```

GET /api/documents
Get user's documents with optional filtering.

****Query Parameters:****

- `document\_type` (optional): Filter by document type
- `limit` (optional): Number of documents to return (default: 10)
- `offset` (optional): Number of documents to skip (default: 0)

GET /api/documents/{document_id}
Get a specific document by ID.

PUT /api/documents/{document_id}
Update a specific document.

DELETE /api/documents/{document_id}
Delete a specific document.

Job Management

```
#### GET /api/jobs
Get discovered jobs with optional filtering.

**Query Parameters:**
- `status` (optional): Filter by job status
- `company` (optional): Filter by company name
- `location` (optional): Filter by location
- `limit` (optional): Number of jobs to return (default: 10)
- `offset` (optional): Number of jobs to skip (default: 0)

#### GET /api/jobs/{job_id}
Get a specific job by ID.

#### PUT /api/jobs/{job_id}/apply
Mark a job as applied.

#### POST /api/jobs/search
Search jobs with custom criteria.

## Error Handling

All API endpoints return consistent error responses:
```

```
{
  "success": false,
  "message": "Error description",
  "errors": ["Detailed error message"],
  "timestamp": "2024-01-15T10:30:00Z"
}
```

```
### HTTP Status Codes

- `200 OK`: Request successful
- `201 Created`: Resource created successfully
- `400 Bad Request`: Invalid request data
- `401 Unauthorized`: Authentication required
- `403 Forbidden`: Insufficient permissions
- `404 Not Found`: Resource not found
- `429 Too Many Requests`: Rate limit exceeded
- `500 Internal Server Error`: Server error

## Rate Limiting

The API implements rate limiting to ensure fair usage:

- **Authenticated users**: 1000 requests per hour
- **Anonymous users**: 100 requests per hour

Rate limit headers are included in responses:
```

X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 999
X-RateLimit-Reset: 1642248000

```
### SDK Generation
```

OpenAPI specification is available at `/openapi.json` for generating SDKs in various languages

```
### Flutter/Dart SDK
```

`openapi-generator generate -i openapi.json -g dart -o mobile/flutter/lib/api_client`

```
### JavaScript SDK
```

`openapi-generator generate -i openapi.json -g javascript -o frontend/js/api_client`

```
### WebSocket Support
```

Real-time updates are available via WebSocket connections:

```
const ws = new WebSocket('wss://your-domain.com/ws');  
ws.onmessage = (event) => {  
  const data = JSON.parse(event.data);  
  // Handle real-time updates  
};
```

6. Enhanced Docker Configuration

`docker-compose.yml`

```
version: '3.8'  
  
services:  
  app:  
    build: .  
    ports:  
      - "8000:8000"  
    environment:  
      - MODE=web  
      - GEMINI_API_KEY=${GEMINI_API_KEY}  
      - GMAIL_CREDENTIALS=${GMAIL_CREDENTIALS}  
      - DATABASE_URL=${DATABASE_URL}  
      - REDIS_URL=${REDIS_URL}
```



```

    - JWT_SECRET=${JWT_SECRET}
volumes:
  - ./data:/app/data
  - ./logs:/app/logs
depends_on:
  - redis
  - db
restart: unless-stopped

worker:
  build: .
  command: celery -A app.celery worker --loglevel=info
  environment:
    - MODE=worker
    - GEMINI_API_KEY=${GEMINI_API_KEY}
    - GMAIL_CREDENTIALS=${GMAIL_CREDENTIALS}
    - DATABASE_URL=${DATABASE_URL}
    - REDIS_URL=${REDIS_URL}
  volumes:
    - ./data:/app/data
    - ./logs:/app/logs
  depends_on:
    - redis
    - db
  restart: unless-stopped

scheduler:
  build: .
  command: celery -A app.celery beat --loglevel=info
  environment:
    - MODE=scheduler
    - GEMINI_API_KEY=${GEMINI_API_KEY}
    - GMAIL_CREDENTIALS=${GMAIL_CREDENTIALS}
    - DATABASE_URL=${DATABASE_URL}
    - REDIS_URL=${REDIS_URL}
  volumes:
    - ./data:/app/data
    - ./logs:/app/logs
  depends_on:
    - redis
    - db
  restart: unless-stopped

redis:
  image: redis:7-alpine
  ports:
    - "6379:6379"
  volumes:
    - redis_data:/data
  restart: unless-stopped

db:
  image: postgres:15-alpine
  environment:
    - POSTGRES_DB=resume_agent
    - POSTGRES_USER=postgres

```

```
- POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
ports:
- "5432:5432"
volumes:
- postgres_data:/var/lib/postgresql/data
restart: unless-stopped

volumes:
  redis_data:
  postgres_data:
```

Dockerfile

```
FROM python:3.11-slim

# Set environment variables
ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1

# Set work directory
WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    gcc \
    g++ \
    && rm -rf /var/lib/apt/lists/*

# Install Python dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Create necessary directories
RUN mkdir -p /app/data /app/logs

# Copy entrypoint script
COPY entrypoint.sh .
RUN chmod +x entrypoint.sh

# Expose port
EXPOSE 8000

# Run entrypoint script
CMD ["/entrypoint.sh"]
```

entrypoint.sh

```
#!/bin/bash

# Enhanced Resume Agent 3.0 Entrypoint Script
# Handles different modes: web, worker, scheduler

set -e

# Wait for database
echo "Waiting for database..."
while ! nc -z db 5432; do
    sleep 1
done
echo "Database is ready!"

# Wait for Redis
echo "Waiting for Redis..."
while ! nc -z redis 6379; do
    sleep 1
done
echo "Redis is ready!"

# Run migrations
echo "Running database migrations..."
python -m alembic upgrade head

# Start application based on MODE
case "$MODE" in
    "web")
        echo "Starting web server..."
        uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4
        ;;
    "worker")
        echo "Starting Celery worker..."
        celery -A app.celery worker --loglevel=info --concurrency=4
        ;;
    "scheduler")
        echo "Starting Celery scheduler..."
        celery -A app.celery beat --loglevel=info
        ;;
    *)
        echo "Starting web server (default)..."
        uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4
        ;;
esac
```

7. Enhanced Environment Configuration

.env.example

```
# Enhanced Resume Agent 3.0 Environment Configuration

# Application Settings
APP_NAME="Enhanced Resume Agent 3.0"
APP_VERSION="3.0.0"
DEBUG=false
LOG_LEVEL=info

# API Configuration
API_HOST=0.0.0.0
API_PORT=8000
API_WORKERS=4

# Database Configuration
DATABASE_URL=postgresql://postgres:password@db:5432/resume_agent
DATABASE_POOL_SIZE=20
DATABASE_MAX_OVERFLOW=30

# Redis Configuration
REDIS_URL=redis://redis:6379/0

# Google API Configuration
GEMINI_API_KEY=your_gemini_api_key_here
GMAIL_CREDENTIALS=your_gmail_credentials_json_here

# Authentication
JWT_SECRET=your_super_secret_jwt_key_here
JWT_ALGORITHM=HS256
JWT_EXPIRATION_HOURS=24

# CORS Configuration
CORS_ORIGINS=*
CORS_METHODS=GET,POST,PUT,DELETE,OPTIONS
CORS_HEADERS=*

# Rate Limiting
RATE_LIMIT_REQUESTS=1000
RATE_LIMIT_WINDOW=3600

# File Storage
STORAGE_TYPE=local
STORAGE_PATH=/app/data/storage

# Logging
LOG_FORMAT=json
LOG_FILE=/app/logs/app.log

# Monitoring
SENTRY_DSN=your_sentry_dsn_here
PROMETHEUS_ENABLED=true

# PWA Configuration
PWA_ENABLED=true
PWA_CACHE_VERSION=3.0.0
```

```
# Mobile App Configuration
MOBILE_APP_SCHEME=resumeagent
MOBILE_DEEP_LINK_DOMAIN=resumeagent.com
```

This comprehensive enhancement transforms your repository into a production-ready, cross-platform AI agent that can be deployed as a web application, PWA, or mobile app. The structure follows industry best practices and provides a solid foundation for scaling across multiple platforms while maintaining code quality and security ^{[1] [2] [3] [4] [5] [6] [7] [8] [9] [10]}.

✱

1. https://langchain-ai.github.io/langgraph/concepts/application_structure/
2. https://github.com/chonhan/flask_restapi_clean_architecture
3. <https://www.gettingstarted.ai/langgraph-tutorial-with-example/>
4. <https://www.datacamp.com/tutorial/langgraph-tutorial>
5. <https://realpython.com/langgraph-python/>
6. <https://learn.microsoft.com/en-us/microsoft-edge/progressive-web-apps/how-to/>
7. <https://blog.codemagic.io/rest-api-in-flutter/>
8. <https://www.geeksforgeeks.org/python/fastapi-openapi/>
9. <https://www.speakeasy.com/openapi/frameworks/fastapi>
10. <https://fastapi.tiangolo.com/how-to/extending-openapi/>